VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Name : Gowdham B

RegNo : 23MIS0300

Course Code : ISWE406P

Course Name : Agile Development Process and Devops Lab

Slot : L3+L4

Patient Records Management System

Problem Statement: Ensure patient records are stored and retrieved accurately without data inconsistencies before deployment.

Tasks:

- Develop patient record management logic
- Write JUnit tests for: o Data validation (missing/incorrect fields) o Record retrieval accuracy
- CI/CD pipeline for automated testing and deployment
- Deploy on Kubernetes Real-world relevance: Hospital patient record systems

App.java code:

```
package com.demo;


public class App {


  static class Patient {
    int id;
    String name;


    Patient(int id, String name) {
      this.id = id;
      this.name = name;
    }
  }


  Patient[] p = new Patient[10];
  int count = 0;


  public boolean addrecord(int id, String name) {
```

```
        if (id <= 0 || name == null || name.isEmpty()) {
            return false;
        }
        p[count++] = new Patient(id, name);
        return true;
    }

    public Patient getrecord(int id) {
        for (int i = 0; i < count; i++) {
            if (p[i].id == id) {
                return p[i];
            }
        }
        return null;
    }

    public static void main(String[] args) {
        System.out.print("App is running...");
    }

}
```

AppTest.java code:

```
package com.demo;

import static org.junit.jupiter.api.Assertions.*;
```

```
import org.junit.jupiter.api.Test;

public class AppTest {
   @Test
   public void validinput() {
      App obj = new App();
      assertTrue(obj.addrecord(1, "GB"));
      assertFalse(obj.addrecord(0, ""));
   }

   @Test
   public void retrieve() {
      App obj = new App();
      obj.addrecord(1, "HH");
      App.Patient o = obj.getrecord(1);
      assertEquals("HH", o.name);
      assertFalse("PN".equals(o.name));
   }
}
```

pom.xml code:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
<groupId>com.demo</groupId>
<artifactId>test-java-app</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>test-java-app</name>

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <junit.version>5.10.1</junit.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
```

```
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
        <configuration>
          <useModulePath>false</useModulePath>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.4.1</version>
        <configuration>
          <archive>
            <manifest>
              <mainClass>App</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
```

```
</project>
```

Dockerfile code:

```
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

deployment.yaml code:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app-deployment
  labels:
    app: java-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
```

```
    containers:
      - name: java-app
        image: gowdhamb/java-app:latest
        imagePullPolicy: Always
        ports:
          - containerPort: 8080
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"


---
apiVersion: v1
kind: Service
metadata:
  name: java-app-service
  labels:
    app: java-app
spec:
  type: NodePort
  selector:
    app: java-app
  ports:
```

- protocol: TCP

  port: 8080

  targetPort: 8080

  nodePort: 30080

Outputs:

Initially after writing the Java code for main file and test file we compile for correcting errors. Once its done we use “**mvn clean package**” command for creating **jar** file and also verifying and validating the main and test codes.

```
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< com.demo:test-java-app >-----------------------
[INFO] Building test-java-app 1.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ test-java-app ---
[INFO] Deleting C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac\target
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ test-java-app ---
[INFO] skip non existing resourceDirectory C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac\src\main\resources
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ test-java-app ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 1 source file with javac [debug target 17] to target\classes
[WARNING] system modules path not set in conjunction with -source 17
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.055 s -- in com.demo.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ test-java-app ---
[INFO] Building jar: C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac\target\test-java-app-1.0.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.051 s
[INFO] Finished at: 2026-04-05T20:03:32+05:30
[INFO] ------------------------------------------------------------------------
```

Github

Github:

Now once we get the build success, we push all the codes, dependencies and configuration files into the github repository.

```
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> git add .
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst', LF will be replaced by CRLF th
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst', LF will be replaced by CRLF the
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst', LF will be replaced by
s it
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst', LF will be replaced by C
it
warning: in the working copy of 'target/surefire-reports/TEST-com.demo.AppTest.xml', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> git commit -m "Initialized successfully"
[main (root-commit) e40a7d9] Initialized successfully
 16 files changed, 245 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 deployment.yaml
 create mode 100644 pom.xml
 create mode 100644 src/main/java/com/demo/App.java
 create mode 100644 src/test/java/com/demo/AppTest.java
 create mode 100644 target/classes/com/demo/App$Patient.class
 create mode 100644 target/classes/com/demo/App.class
 create mode 100644 target/maven-archiver/pom.properties
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
 create mode 100644 target/surefire-reports/TEST-com.demo.AppTest.xml
 create mode 100644 target/surefire-reports/com.demo.AppTest.txt
 create mode 100644 target/test-classes/com/demo/AppTest.class
 create mode 100644 target/test-java-app-1.0.jar
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> git remote add origin "https://github.com/Gowdham-B/agilelabmidtermprac.git"
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> git push -u origin  main
Enumerating objects: 42, done.
Counting objects: 100% (42/42), done.
Delta compression using up to 16 threads
Compressing objects: 100% (23/23), done.
Writing objects: 100% (42/42), 9.52 KiB | 974.00 KiB/s, done.
Total 42 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Gowdham-B/agilelabmidtermprac.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Jenkins:

Once the codes and other files are pushed to the github, we now open Jenkins application and create a job/task named "**Java-Full-Scenario**" with the option as pipeline. Now we scroll down and we put the pipeline script and save the configuration.

Pipeline script:

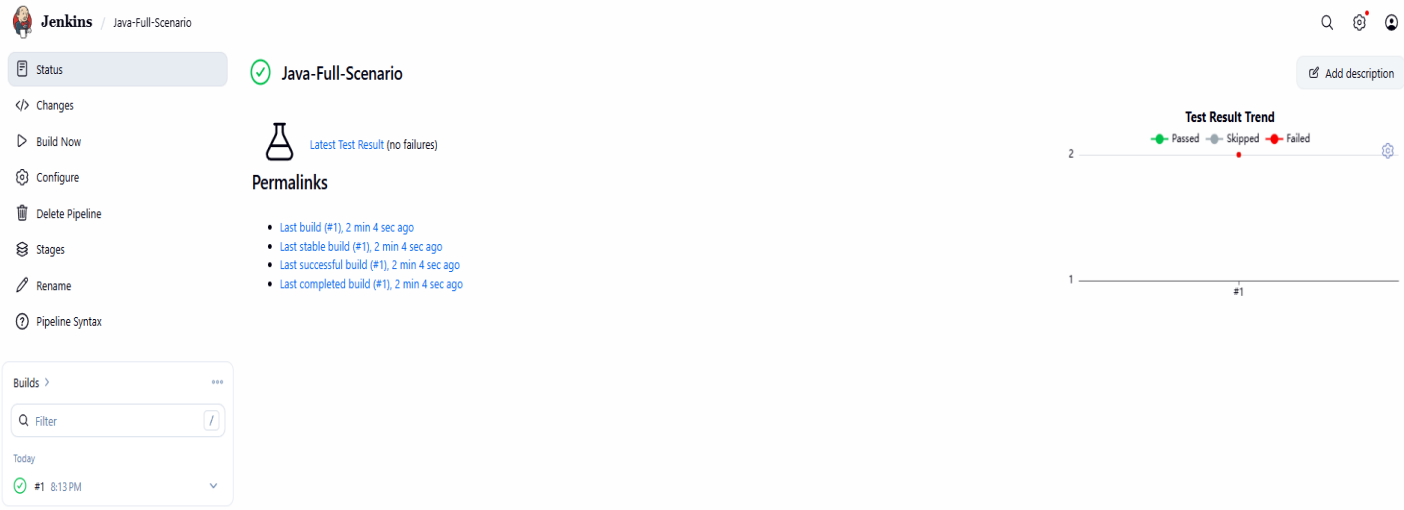
```
pipeline {
 agent any
 environment {
```

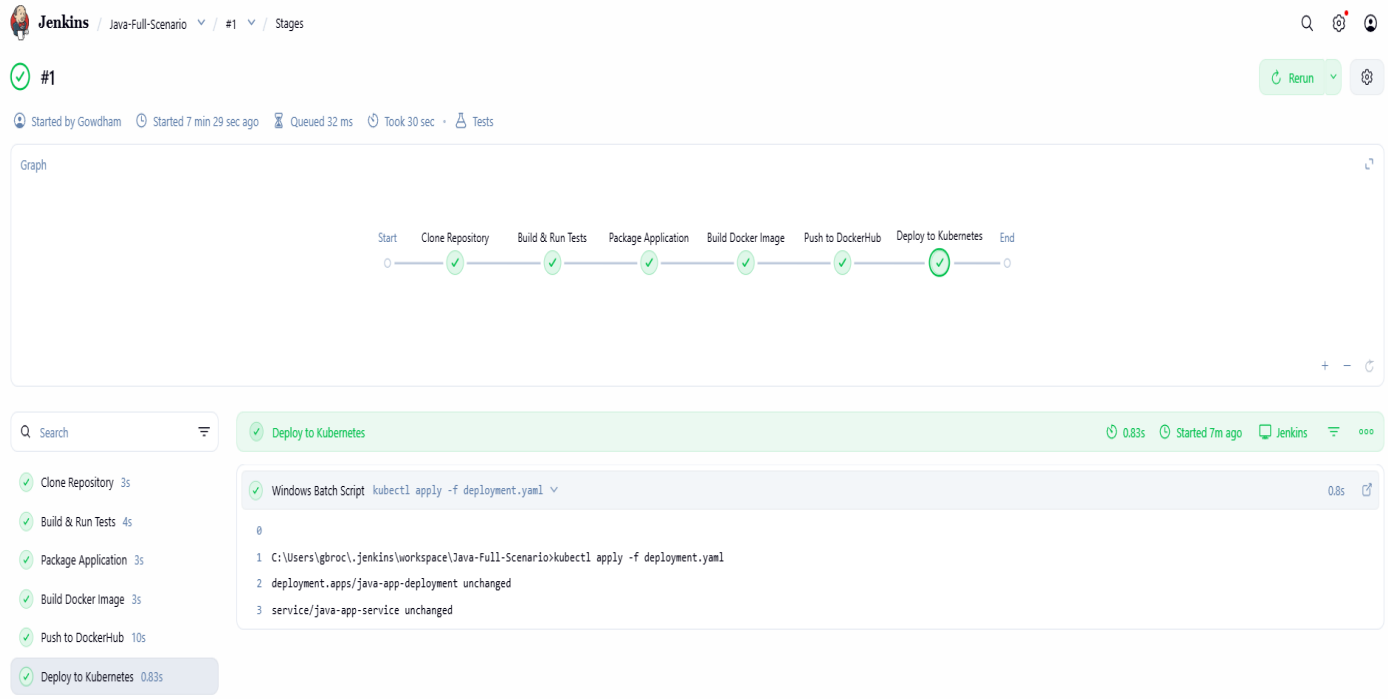
```
DOCKER_IMAGE = "gowdhamb/java-full-scenario:latest"
}
stages {
stage('Clone Repository') {
steps {
git branch: 'main',
url: 'https://github.com/Gowdham-B/agilelabmidtermprac.git'
}
}
stage('Build & Run Tests') {
steps {
bat 'mvn clean test'
}
post {
always {
junit 'target/surefire-reports/*.xml'
}
}
}
stage('Package Application') {
steps {
bat 'mvn package -DskipTests'
}
}
stage('Build Docker Image') {
steps {
```

```
bat "docker build -t %DOCKER_IMAGE% ."
}
}
stage('Push to DockerHub') {
steps {
withCredentials([usernamePassword(
credentialsId: 'dockerhub-creds',
usernameVariable: 'DOCKER_USER',
passwordVariable: 'DOCKER_PASS'
)]) {
bat """
docker login -u %DOCKER_USER% -p %DOCKER_PASS%
docker push %DOCKER_IMAGE%
"""
}
}
}
stage('Deploy to Kubernetes') {
steps {
bat 'kubectl apply -f deployment.yaml'
}
}
}
}
```

Now after saving the pipeline script, we run the build now option to automate the docker image creation and deploying the containers in the Kubernetes.

Here itself we have mentioned the required commands to create docker image and deploying the files in Kubernetes. So once the build is successful then it means the Docker image is created and it is deployed in Kubernetes.



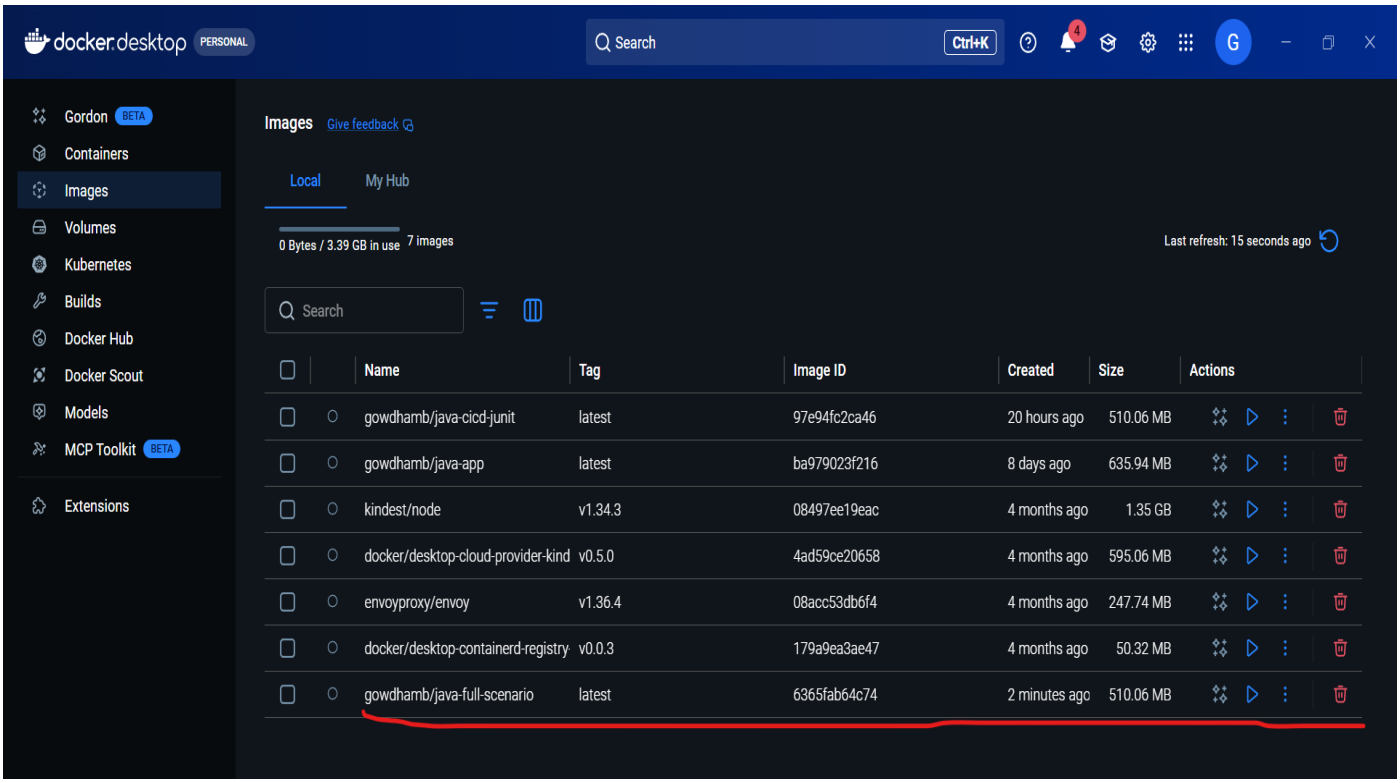


```
ca6b40ab0beb: Pushed
961131a7e056: Pushed
latest: digest: sha256:6365fab64c742dc2151d01ff43296e4fad9bf554a21249413d076e5331e09882 size: 856
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Kubernetes)
[Pipeline] bat

C:\Users\gbroc\.jenkins\workspace\Java-Full-Scenario>kubectl apply -f deployment.yaml
deployment.apps/java-app-deployment unchanged
service/java-app-service unchanged
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Docker Desktop:

Here in the below image, we can see the created Docker image.



Kubernetes:

Now since the Kubernetes deployment is not accessible through my Docker Desktop, I have used some commands to show the successful deployment of images in the Kubernetes.

```
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> kubectl get pods
NAME                                    READY   STATUS    RESTARTS        AGE
java-app-deployment-6575dc6cb8-vktfr    1/1     Running   12 (4h10m ago)  8d
java-app-deployment-6575dc6cb8-vwdcr    1/1     Running   12 (4h10m ago)  8d
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> kubectl get nodes
NAME                   STATUS   ROLES           AGE   VERSION
desktop-control-plane  Ready    control-plane   8d    v1.34.3
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac> kubectl get svc
NAME               TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
java-app-service   NodePort    10.96.235.129   <none>        8080:30080/TCP   8d
kubernetes         ClusterIP   10.96.0.1       <none>        443/TCP          8d
PS C:\Users\gbroc\OneDrive\Documents\agilelabmidtermprac>
```

Final report:

Hence the **Patient Record Management** system is implemented successfully in Kubernetes through Docker Desktop via Jenkins pipeline.